

Output

The first line contains only one integer h , the maximum number of edges Mohana and John can add.

Each of the next h lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edge you add each time.

If there are multiple correct answers, you can print any one of them.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3 2
5 4
2 1
4 3
4 3
1 4
```

EXPECTED OUTPUT

```
1
1 5
```

Test Case 2

INPUT (STDIN)

```
5 3 2
5 3
2 3
2 3
2 3
2 4
```

EXPECTED OUTPUT

```
1
1 2
4 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m1--)
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

500

Test Case 3

NLOC

30

```
1 #include <stdio.h>
2
3 #define MAXN 100005
4
5 int par1[MAXN];
6 int par2[MAXN];
7 int eu[MAXN];
8 int ev[MAXN];
9
10 void init(int par[], int n) {
11     int i;
12     for (i = 1; i <= n; i++) par[i] = i;
13 }
14
15 int find(int par[], int x) {
16     while (par[x] != x) {
17         par[x] = par[par[x]];
18         x = par[x];
19     }
20     return x;
21 }
22
23 void unite(int par[], int a, int b) {
24     a = find(par, a);
25     b = find(par, b);
```

```
26     if (a != b) par[a] = b;
```

```
27 }
```

```
28
```

```
29 int main() {
```

main | CH.SC.U4CSE25225 | Tree19.c

Ln 57, Col 1 Spaces: 4 UTF-8 CRLF C

```
30     int n, m1, m2, i;
```

```
31     int h = 0;
```

```
32     if (scanf("%d %d %d", &n, &m1, &m2) != 3) return 0;
```

```
33     init(par1, n);
```

```
34     init(par2, n);
```

```
35     for (i = 0; i < m1; i++) {
```

```
36         int u, v;
```

```
37     scanf("%d %d", &u, &v);
38     unite(par1, u, v);
39 }
40 for (i = 0; i < m2; i++) {
41     int u, v;
42     scanf("%d %d", &u, &v);
43     unite(par2, u, v);
44 }
45 for (i = 2; i <= n; i++) {
46     if (find(par1, i) != find(par1, 1) && find(par2, i) != find(par2, 1)) {
47         unite(par1, i, 1);
```

```
48         unite(par2, i, 1);
49         eu[h] = 1;
50         ev[h] = i;
51         h++;
52     }
53 }
54 printf("%d\n", h);
55 for (i = 0; i < h; i++) printf("%d %d\n", eu[i], ev[i]);
56 return 0;
57}
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 3 2

5 4

2 1

4 3

4 3

1 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree19.exe

1

1 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5 3 2

5 3

2 3

2 3

2 3

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree19.exe

1

1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>